

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.

Scenario Analysis

The university stores student and course information in a single table. Each time a student enrolls in a new course, the student's personal details, course details, and faculty information are repeated. This creates unnecessary data redundancy, wastes storage space, and makes the database difficult to maintain. If any information changes, multiple records must be updated, increasing the chance of inconsistency. Therefore, the table should be normalized to improve efficiency and maintain data integrity.

Existing Table (Unnormalized Form)

StudentID	StudentName	Department	CourseID	CourseName	FacultyID	FacultyName	FacultyPhone
S101	Arun	CSE	C201	DBMS	F01	Ravi	9876543210
S101	Arun	CSE	C202	CN	F02	Kumar	9876501234
S102	Priya	IT	C201	DBMS	F01	Ravi	9876543210

Normalization Issues Identified

The above table has the following problems:

- Student details are repeated for every course enrolled.
- Course details are repeated for different students.
- Faculty information is stored multiple times.
- Updating faculty phone numbers requires modifying several rows (**Update Anomaly**).
- A new course cannot be inserted until a student enrolls (**Insertion Anomaly**).
- Deleting the last student enrolled in a course may remove the course details (**Deletion Anomaly**).
- The table contains **partial dependencies** because student information depends only on **StudentID**, while course information depends only on **CourseID**.
- The table also contains **transitive dependencies** because **FacultyPhone** depends on **FacultyID**, not directly on the primary key.

Normalization Process

First Normal Form (1NF)

A table is in **1NF** if all attributes contain only atomic values and there are no repeating groups.

Enrollment Table

StudentID	CourseID
S101	C201
S101	C202
S102	C201

The table now contains only atomic values, but redundancy still exists.

Second Normal Form (2NF)

A table is in **2NF** if it is already in 1NF and every non-key attribute is fully dependent on the whole primary key.

The composite key in the Enrollment table is **(StudentID, CourseID)**. Student details depend only on **StudentID**, while course details depend only on **CourseID**. Therefore, they are separated into different tables.

Student Table

StudentID	StudentName	Department
S101	Arun	CSE
S102	Priya	IT

Course Table

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	CN	F02

Enrollment Table

StudentID	CourseID
S101	C201
S101	C202
S102	C201

This removes all **partial dependencies**.

Third Normal Form (3NF)

A table is in **3NF** if it is already in 2NF and contains no transitive dependency. Faculty details depend on **FacultyID**, so they are moved into a separate table.

Faculty Table

FacultyID	FacultyName	FacultyPhone
F01	Ravi	9876543210
F02	Kumar	9876501234

Updated Course Table

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	CN	F02

Final 3NF Database Schema

Student

Attribute	Description
StudentID (PK)	Primary Key
StudentName	Student Name
Department	Department

Faculty

Attribute	Description
FacultyID (PK)	Primary Key
FacultyName	Faculty Name
FacultyPhone	Contact Number

Course

Attribute	Description
CourseID (PK)	Primary Key
CourseName	Course Name
FacultyID (FK)	References Faculty

Enrollment

Attribute	Description
StudentID (FK)	References Student
CourseID (FK)	References Course
Primary Key	(StudentID, CourseID)

Benefits of 3NF

- Eliminates data redundancy.
- Removes insertion, update, and deletion anomalies.
- Improves data consistency and integrity.
- Reduces storage requirements.
- Simplifies database maintenance.
- Makes querying and updating data more efficient.
- Supports future scalability and expansion.

Justification

The redesigned database satisfies **Third Normal Form (3NF)** because every non-key attribute depends only on the primary key and not on another non-key attribute. Separating Student, Course, Faculty, and Enrollment information removes redundancy and anomalies while preserving relationships through primary and foreign keys. This design improves reliability, maintainability, and overall database performance.

Conclusion

The original single-table design resulted in repeated data, redundancy, and several anomalies. By normalizing the database up to **Third Normal Form (3NF)**, the data is organized into **Student**, **Faculty**, **Course**, and **Enrollment** tables. The redesigned schema eliminates redundancy, ensures data integrity, improves storage efficiency, and provides a robust and scalable database structure suitable for a university management system.

Q2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

Scenario Analysis

A company maintains information about its employees and departments in a relational database. In many organizations, an employee may work in more than one department, such as participating in cross-functional projects or assisting multiple teams. This creates a **many-to-many relationship** between employees and departments.

To represent this relationship efficiently, the database should use three normalized tables:

- **Employee** – Stores employee details.
- **Department** – Stores department details.
- **Employee_Department** – Stores the mapping between employees and departments.

The objective is to retrieve employees who are assigned to **more than one department** using SQL while ensuring data accuracy, integrity, and efficiency.

Database Schema

Employee Table

EmployeeID	EmployeeName	Designation	Salary
E101	Arun	Software Engineer	50000
E102	Priya	System Analyst	65000
E103	Rahul	Database Administrator	70000

Primary Key: EmployeeID

Department Table

DepartmentID	DepartmentName
D01	Human Resources
D02	Information Technology
D03	Finance
D04	Marketing

Primary Key: DepartmentID

Employee_Department Table

EmployeeID	DepartmentID
E101	D01
E101	D02
E102	D02
E103	D02
E103	D03

Composite Primary Key: (EmployeeID, DepartmentID)

Foreign Keys:

- EmployeeID → Employee (EmployeeID)
- DepartmentID → Department (DepartmentID)

Problem Identification

From the above data:

- **Arun (E101)** works in **Human Resources** and **Information Technology**.
- **Priya (E102)** works only in **Information Technology**.
- **Rahul (E103)** works in **Information Technology** and **Finance**.

Therefore, the database should return **Arun** and **Rahul**, as they work in more than one department.

SQL Query to Retrieve Employees Working in Multiple Departments

```
mysql> SELECT
->     E.EmployeeID,
->     E.EmployeeName,
->     COUNT(ED.DepartmentID) AS TotalDepartments
-> FROM Employee E
-> JOIN Employee_Department ED
-> ON E.EmployeeID = ED.EmployeeID
-> GROUP BY E.EmployeeID, E.EmployeeName
-> HAVING COUNT(ED.DepartmentID) > 1;
```

Explanation of the Query

- The **JOIN** operation combines the Employee and Employee_Department tables using EmployeeID.
- **GROUP BY** groups all department records belonging to the same employee.
- **COUNT ()** calculates the total number of departments assigned to each employee.
- **HAVING COUNT(DepartmentID) > 1** filters only those employees who are associated with more than one department.
- The result displays the employee ID, employee name, and the number of departments in which each employee works.

Expected Output

EmployeeID	EmployeeName	TotalDepartments
E101	Arun	2
E103	Rahul	2

This output confirms that Arun and Rahul are working in multiple departments.

SQL Query to Display Employee with Department Names

```
mysql> SELECT E.EmployeeID, E.EmployeeName, D.DepartmentName FROM Employee E JOIN Employee_Department ED ON E.EmployeeID = ED.EmployeeID JOIN Department D  
-> ON ED.DepartmentID = D.DepartmentID WHERE E.EmployeeID IN (SELECT EmployeeID  
-> FROM Employee_Department GROUP BY EmployeeID HAVING COUNT(DepartmentID) > 1) ORDER BY E.EmployeeID;
```

Expected Output

EmployeeID	EmployeeName	DepartmentName
E101	Arun	Human Resources
E101	Arun	Information Technology
E103	Rahul	Information Technology
E103	Rahul	Finance

This query provides detailed information by displaying both the employee names and the departments in which they work.

Justification of the Approach

The proposed solution follows relational database design principles by separating employee, department, and relationship data into different tables. This eliminates redundancy and ensures data consistency.

The **JOIN** operation efficiently combines related tables without duplicating data. The **GROUP BY** clause groups records employee-wise, while the **COUNT()** function determines the number of departments assigned to each employee. The **HAVING** clause filters only those employees whose department count is greater than one.

This approach is efficient because:

- It uses normalized tables.
- It avoids duplicate employee information.
- It retrieves only the required records.
- It maintains referential integrity using primary and foreign keys.
- It is scalable and suitable for large organizational databases.

Advantages

- Eliminates data redundancy through normalization.
- Correctly handles many-to-many relationships.
- Maintains referential integrity using primary and foreign keys.
- Efficiently retrieves employees working in multiple departments.
- Improves query performance using JOIN and GROUP BY.
- Produces accurate and meaningful reports for management.
- Supports future expansion by allowing employees to be assigned to additional departments without modifying the database structure.

Conclusion

The designed database schema and SQL queries successfully identify employees working in multiple departments. By using normalized tables, **JOIN**, **GROUP BY**, **COUNT ()**, and **HAVING**, the solution efficiently retrieves accurate information while maintaining data integrity and minimizing redundancy. This approach is reliable, scalable, and well suited for real-world company database systems, making it an ideal solution for the given scenario.

Q3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Scenario Analysis

A relation contains multiple attributes with several functional dependencies. The objective is to identify the **candidate key(s)**, verify whether the relation satisfies **Boyce-Codd Normal Form (BCNF)**, and, if necessary, decompose the relation into smaller relations while preserving data integrity and reducing redundancy.

BCNF is a stronger form of normalization than 3NF. A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, **X must be a super key**.

Given Relation

Let the relation be:

R (StudentID, StudentName, CourseID, CourseName, FacultyID, FacultyName)

Functional Dependencies

1. $\text{StudentID} \rightarrow \text{StudentName}$
2. $\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$
3. $\text{FacultyID} \rightarrow \text{FacultyName}$
4. $\text{StudentID}, \text{CourseID} \rightarrow \text{StudentName}, \text{CourseName}, \text{FacultyID}, \text{FacultyName}$

Determining the Candidate Key

To identify the candidate key, compute the attribute closure.

Closure of (StudentID, CourseID)

$(\text{StudentID}, \text{CourseID})^+$

- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$
- $\text{FacultyID} \rightarrow \text{FacultyName}$

Therefore,

$(\text{StudentID}, \text{CourseID})^+ =$

$\{\text{StudentID}, \text{StudentName}, \text{CourseID}, \text{CourseName}, \text{FacultyID}, \text{FacultyName}\}$

Since all attributes of the relation are obtained, **(StudentID, CourseID)** is a **Candidate Key**.

No smaller combination of attributes can determine all attributes of the relation.

Candidate Key = (StudentID, CourseID)

Checking BCNF

A relation is in BCNF only if the determinant of every functional dependency is a super key.

Functional Dependency Analysis

Functional Dependency	Super Key?	BCNF Status
$\text{StudentID} \rightarrow \text{StudentName}$	No	Violates BCNF
$\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$	No	Violates BCNF
$\text{FacultyID} \rightarrow \text{FacultyName}$	No	Violates BCNF
$\text{StudentID}, \text{CourseID} \rightarrow \text{All Attributes}$	Yes	Satisfies BCNF

Observation

The first three functional dependencies violate BCNF because their determinants (**StudentID**, **CourseID**, and **FacultyID**) are **not super keys**.

Therefore, the given relation **does not satisfy BCNF**.

BCNF Decomposition

To remove the violations, decompose the relation into smaller relations.

Student Relation

StudentID	StudentName
S101	Arun
S102	Priya

Functional Dependency

StudentID $\rightarrow$ StudentName

Primary Key: **StudentID**

Faculty Relation

FacultyID	FacultyName
F01	Ravi
F02	Kumar

Functional Dependency

FacultyID $\rightarrow$ FacultyName

Primary Key: **FacultyID**

Course Relation

CourseID	CourseName	FacultyID
C201	DBMS	F01
C202	Computer Networks	F02

Functional Dependency

CourseID $\rightarrow$ CourseName, FacultyID

Primary Key: **CourseID**

Foreign Key: **FacultyID**

Enrollment Relation

StudentID	CourseID
S101	C201
S101	C202
S102	C201

Primary Key: (**StudentID, CourseID**)

Foreign Keys:

- StudentID references Student
- CourseID references Course

Final BCNF Relations

Student

(StudentID **PK**, StudentName)

Faculty

(FacultyID **PK**, FacultyName)

Course

(CourseID **PK**, CourseName, FacultyID **FK**)

Enrollment

(StudentID **FK**, CourseID **FK**, Primary Key(StudentID, CourseID))

Advantages of BCNF Decomposition

- Eliminates data redundancy.
- Removes update, insertion, and deletion anomalies.
- Ensures every determinant is a candidate key.
- Improves data consistency and integrity.
- Simplifies database maintenance.
- Makes the database easier to scale and manage.
- Preserves relationships using primary and foreign keys.

Justification

The original relation violated BCNF because several functional dependencies had determinants that were not super keys. By decomposing the relation into **Student**, **Faculty**, **Course**, and **Enrollment**, every functional dependency is associated with a relation where the determinant is a primary key. This eliminates redundancy, prevents anomalies, and maintains referential integrity through primary and foreign keys.

Conclusion

The candidate key for the given relation is (**StudentID**, **CourseID**). The original relation does not satisfy **BCNF** because **StudentID**, **CourseID**, and **FacultyID** are determinants but not super keys. After decomposition into **Student**, **Faculty**, **Course**, and **Enrollment** relations, every relation satisfies BCNF. The redesigned schema removes redundancy, preserves data integrity, supports efficient querying, and provides a well-structured database suitable for real-world applications.

Q4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Scenario Analysis

A banking system stores highly sensitive information such as customer details, account balances, transactions, and loan records. Different users of the banking system have different responsibilities and should not have unrestricted access to all data.

For example:

- **Customers** should view only their own account information.
- **Bank Employees** should access customer details required for banking operations but should not modify sensitive account information unnecessarily.
- **Managers** should have access to reports and customer details for decision-making.
- **Administrators** should have complete access to maintain the database.

To ensure confidentiality, integrity, and controlled access, the database should use **Views** to restrict data visibility and **Integrity Constraints** to maintain valid and consistent data.

Database Schema

Customer Table

CustomerID	CustomerName	Phone	Address
C101	Arun	9876543210	Chennai
C102	Priya	9876501234	Vellore

Account Table

AccountNo	CustomerID	AccountType	Balance
A1001	C101	Savings	50000
A1002	C102	Current	75000

Transaction Table

TransactionID	AccountNo	Amount	TransactionType
T101	A1001	5000	Deposit
T102	A1002	2000	Withdrawal

Security Requirements

The banking system must:

- Allow customers to view only their own account details.
- Allow employees to access customer and account information without exposing confidential data unnecessarily.
- Allow managers to generate reports.
- Prevent invalid data entry.
- Maintain data consistency using integrity constraints.

Designing Database Views

Customer View

Customers should only see their own account information.

```
mysql> CREATE VIEW Customer_View AS SELECT CustomerID, CustomerName,  
-> AccountNo, AccountType, Balance FROM Customer JOIN Account  
-> ON Customer.CustomerID = Account.CustomerID;
```

This view hides unnecessary information such as phone number, address, and transaction history.

Employee View

Employees require customer and account details to perform banking operations.

```
mysql> CREATE VIEW Employee_View AS  
-> SELECT CustomerID,  
-> CustomerName,  
-> AccountNo,  
-> AccountType,  
-> Balance  
-> FROM Customer  
-> JOIN Account  
-> ON Customer.CustomerID = Account.CustomerID;
```

Employees can view customer account details without accessing administrative information.

Manager View

Managers need transaction details for monitoring and reporting.

```
mysql> CREATE VIEW Manager_View AS  
-> SELECT  
-> TransactionID,  
-> AccountNo,  
-> Amount,  
-> TransactionType  
-> FROM Transaction;
```

This view helps managers generate reports while maintaining customer privacy.

Integrity Constraints

Primary Key Constraint

Ensures that each record is unique.

```
mysql> CREATE TABLE Customer
-> (
-> CustomerID VARCHAR(10) PRIMARY KEY,
-> CustomerName VARCHAR(50),
-> Phone VARCHAR(15),
-> Address VARCHAR(100)
-> );
Query OK, 0 rows affected (0.12 sec)
```

Foreign Key Constraint

Maintains the relationship between Customer and Account tables.

```
mysql> CREATE TABLE Account
-> (
-> AccountNo VARCHAR(10) PRIMARY KEY,
-> CustomerID VARCHAR(10),
-> AccountType VARCHAR(20),
-> Balance DECIMAL(10,2),
-> FOREIGN KEY (CustomerID)
-> REFERENCES Customer(CustomerID)
-> );
Query OK, 0 rows affected (0.10 sec)
```

NOT NULL Constraint

Ensures mandatory fields always contain values.

```
mysql> CustomerName VARCHAR(50) NOT NULL
->
```

UNIQUE Constraint

Ensures duplicate phone numbers are not stored.

```
mysql> Phone VARCHAR(15) UNIQUE;
```

CHECK Constraint

Ensures account balance cannot become negative.

Only valid account types are accepted.

```
mysql> Balance DECIMAL(10,2)
-> CHECK (Balance >= 0)
-> Another example:
-> CHECK (AccountType IN ('Savings','Current'))
```

DEFAULT Constraint

Assigns a default balance when a new account is created.

```
mysql> Balance DECIMAL(10,2)  
-> DEFAULT 0
```

How Views Improve Security

- Customers can access only their own account details.
- Employees are restricted from viewing confidential administrative data.
- Managers receive summarized information required for monitoring.
- Sensitive information is hidden from unauthorized users.
- Reduces the risk of accidental modification of critical data.

How Constraints Improve Data Integrity

- Primary Key prevents duplicate records.
- Foreign Key maintains valid relationships between tables.
- NOT NULL ensures mandatory information is entered.
- UNIQUE prevents duplicate values.
- CHECK validates business rules.
- DEFAULT automatically assigns valid values when needed.

Advantages

- Protects confidential banking information.
- Prevents unauthorized access.
- Maintains data accuracy and consistency.
- Eliminates invalid or duplicate records.
- Improves database security.
- Simplifies database administration.
- Supports reliable transaction processing.
- Ensures compliance with banking data management standards.

Justification

The proposed design uses **Views** to implement role-based access control, ensuring that each type of user can access only the information necessary for their responsibilities. At the same time, **integrity constraints** enforce business rules, prevent invalid data entry, and preserve relationships between tables. Together, these mechanisms improve both **database security** and **data integrity**, making the banking system reliable, secure, and easy to maintain.

Conclusion

The banking database is secured by implementing **Views** and **Integrity Constraints**. Views restrict access based on user roles, while constraints such as **Primary Key, Foreign Key, NOT NULL, UNIQUE, CHECK, and DEFAULT** ensure that only valid and consistent data is stored. This design protects sensitive customer information, maintains referential integrity, prevents data anomalies, and provides a secure, efficient, and scalable banking database suitable for real-world applications.

Q5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

Scenario Analysis

A sales database stores customer, product, salesperson, and order details in a single table. Since multiple products can be purchased by the same customer, customer and salesperson information is repeatedly stored for every order. This creates **data redundancy**, increases storage requirements, and causes anomalies during insertion, update, and deletion operations.

The objective is to analyze the existing schema, identify normalization issues, and redesign the database up to **Third Normal Form (3NF)** or **Boyce-Codd Normal Form (BCNF)** to improve data integrity and eliminate redundancy.

Existing Unnormalized Sales Table

Order ID	CustomerID	CustomerName	ProductID	ProductName	SalesPersonID	SalesPersonName	Quantity
O101	C101	Arun	P101	Laptop	S01	Ravi	2
O102	C101	Arun	P102	Mouse	S01	Ravi	1
O103	C102	Priya	P101	Laptop	S02	Kumar	1

Problems in the Existing Schema

The above table has several normalization issues:

- Customer information is repeated for every order.
- Product details are repeated whenever the product is sold.
- Salesperson details are stored multiple times.
- Updating customer or salesperson information requires modifying multiple rows (**Update Anomaly**).
- A new product or customer cannot be inserted until an order is placed (**Insertion Anomaly**).
- Deleting the last order of a customer may remove customer information (**Deletion Anomaly**).
- The table contains **partial dependencies** and **transitive dependencies**, violating higher normal forms.

Normalization Process

First Normal Form (1NF)

A relation is in **1NF** if:

- Every column contains atomic values.
- There are no repeating groups.
- Each record is unique.

The given table already contains atomic values, so it satisfies **1NF**.

Second Normal Form (2NF)

A relation is in **2NF** if:

- It is already in 1NF.
- Every non-key attribute is fully dependent on the entire primary key.

Separate customer, product, and order information into different tables.

Customer Table

CustomerID	CustomerName
C101	Arun
C102	Priya

Primary Key: **CustomerID**

Product Table

ProductID	ProductName
P101	Laptop
P102	Mouse

Primary Key: **ProductID**

SalesPerson Table

SalesPersonID	SalesPersonName
S01	Ravi
S02	Kumar

Primary Key: **SalesPersonID**

Orders Table

OrderID	CustomerID	ProductID	SalesPersonID	Quantity
O101	C101	P101	S01	2
O102	C101	P102	S01	1
O103	C102	P101	S02	1

Primary Key: **OrderID**

Foreign Keys:

- CustomerID → Customer (CustomerID)
- ProductID → Product (ProductID)
- SalesPersonID → SalesPerson (SalesPersonID)

This removes **partial dependencies**.

Third Normal Form (3NF)

A relation is in **3NF** if:

- It is already in **2NF**.
- There are **no transitive dependencies**.
- Every non-key attribute depends only on the primary key.

The decomposed tables satisfy 3NF because:

- CustomerName depends only on CustomerID.
- ProductName depends only on ProductID.
- SalesPersonName depends only on SalesPersonID.
- Quantity depends only on OrderID.

Thus, every non-key attribute depends directly on its table's primary key.

BCNF Verification

A relation is in **BCNF** if, for every functional dependency $X \rightarrow Y$, **X must be a super key**.

Functional Dependencies

- CustomerID $\rightarrow$ CustomerName
- ProductID $\rightarrow$ ProductName
- SalesPersonID $\rightarrow$ SalesPersonName
- OrderID $\rightarrow$ CustomerID, ProductID, SalesPersonID, Quantity

In each relation, the determinant is the primary key. Therefore, **all relations satisfy BCNF**.

Final Normalized Schema

Customer

(CustomerID **PK**, CustomerName)

Product

(ProductID **PK**, ProductName)

SalesPerson

(SalesPersonID **PK**, SalesPersonName)

Orders

(OrderID **PK**, CustomerID **FK**, ProductID **FK**, SalesPersonID **FK**, Quantity)

Advantages of Normalization

- Eliminates data redundancy.
- Removes insertion, update, and deletion anomalies.
- Improves data consistency and integrity.
- Reduces storage requirements.
- Simplifies database maintenance.
- Improves query efficiency.
- Supports scalability and future expansion.
- Ensures reliable relationships using primary and foreign keys.

Justification

The original sales database contained repeated customer, product, and salesperson information, leading to redundancy and anomalies. By decomposing the schema into **Customer**, **Product**, **SalesPerson**, and **Orders** tables, each entity stores only relevant information. Primary and foreign keys preserve relationships between tables while ensuring referential integrity. The resulting design satisfies **3NF** and **BCNF**, eliminates redundancy, and provides an efficient and maintainable database structure.

Conclusion

The sales database was successfully analyzed and normalized from an unnormalized structure to **Third Normal Form (3NF)** and verified to satisfy **Boyce-Codd Normal Form (BCNF)**. The redesigned schema eliminates redundancy, prevents insertion, update, and deletion anomalies, improves data integrity, and provides a scalable, efficient, and well-structured database suitable for real-world sales management systems.

Q6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Scenario Analysis

A company maintains a database containing **Customer** and **Order** information. Each customer can place multiple orders, creating a **one-to-many relationship** between the Customer and Orders tables.

The objective is to retrieve meaningful business information such as:

- Customers who place orders frequently.
- Customer order details.
- Total orders placed by each customer.
- Customers who have placed more than a specified number of orders.

This information helps the company identify loyal customers, improve marketing strategies, and reward frequent buyers.

Database Schema

Customer Table

CustomerID	CustomerName	City
C101	Arun	Chennai
C102	Priya	Vellore
C103	Rahul	Bangalore

Primary Key: CustomerID

Orders Table

OrderID	CustomerID	OrderDate	Amount
O101	C101	01-08-2026	5000
O102	C101	02-08-2026	3500
O103	C102	03-08-2026	7000
O104	C101	04-08-2026	4500
O105	C103	05-08-2026	2500

Primary Key: OrderID

Foreign Key: CustomerID references Customer(CustomerID)

Relationship

Customer (1) -----< Orders (Many)

Each customer can place multiple orders, but each order belongs to only one customer.

SQL Query Using JOIN

The following query retrieves customer details along with their order information.

```
mysql> SELECT
->     C.CustomerID,
->     C.CustomerName,
->     O.OrderID,
->     O.OrderDate,
->     O.Amount
-> FROM Customer C
-> INNER JOIN Orders O
-> ON C.CustomerID = O.CustomerID;
```

CustomerID	CustomerName	OrderID	OrderDate	Amount
C101	Arun	0101	2026-08-01	5000.00
C101	Arun	0102	2026-08-02	3500.00
C101	Arun	0104	2026-08-04	4500.00
C102	Priya	0103	2026-08-03	7000.00
C103	Rahul	0105	2026-08-05	2500.00

5 rows in set (0.00 sec)

Explanation

- **INNER JOIN** combines records from Customer and Orders based on the matching CustomerID.
- Only customers who have placed orders are displayed.
- This query provides complete order information for each customer.

SQL Query to Find Frequent Customers Using GROUP BY

```
mysql> SELECT
->     C.CustomerID,
->     C.CustomerName,
->     COUNT(O.OrderID) AS TotalOrders
-> FROM Customer C
-> JOIN Orders O
-> ON C.CustomerID = O.CustomerID
-> GROUP BY C.CustomerID, C.CustomerName
-> HAVING COUNT(O.OrderID) > 2;
```

CustomerID	CustomerName	TotalOrders
C101	Arun	3

1 row in set (0.03 sec)

Explanation

- **GROUP BY** groups all orders belonging to each customer.
- **COUNT ()** counts the total number of orders placed.
- **HAVING** filters customers who placed more than two orders.
- This query identifies frequent or loyal customers.

SQL Query Using Subquery

The following query retrieves customers whose number of orders is greater than two using a subquery.

```
mysql> SELECT
->     CustomerID,
->     CustomerName
-> FROM Customer
-> WHERE CustomerID IN
-> (
->     SELECT CustomerID
->     FROM Orders
->     GROUP BY CustomerID
->     HAVING COUNT(OrderID) > 2
-> );
```

```
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
| C101      | Arun         |
+-----+-----+
1 row in set (0.02 sec)
```

Explanation

- The **subquery** identifies CustomerIDs having more than two orders.
- The outer query retrieves the corresponding customer details.
- This approach separates the filtering logic from the main query, improving readability.

SQL Query to Calculate Total Purchase Amount

```
mysql> SELECT
->     C.CustomerID,
->     C.CustomerName,
->     SUM(O.Amount) AS TotalPurchase
-> FROM Customer C
-> JOIN Orders O
-> ON C.CustomerID = O.CustomerID
-> GROUP BY C.CustomerID, C.CustomerName;
+-----+-----+-----+
| CustomerID | CustomerName | TotalPurchase |
+-----+-----+-----+
| C101      | Arun         | 13000.00      |
| C102      | Priya        | 7000.00        |
| C103      | Rahul        | 2500.00        |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Advantages of Using JOINS and Subqueries

- Retrieves data from multiple related tables efficiently.
- Eliminates redundant data retrieval.
- Helps identify frequent and loyal customers.
- Simplifies report generation for business analysis.
- Supports decision-making through meaningful insights.
- Improves query readability and maintainability.
- Enables efficient analysis of customer purchasing behavior.

Justification

The **JOIN** operation efficiently combines customer and order data using the common CustomerID, avoiding redundant storage and ensuring accurate results. The **GROUP BY** and **HAVING** clauses identify customers based on their order frequency, while the **subquery** provides an alternative and structured method for filtering frequent customers. These SQL techniques are widely used in real-world business applications for customer analysis, sales reporting, and marketing decisions.

Conclusion

The proposed SQL queries successfully retrieve meaningful information from the Customer and Orders tables using **JOINS** and **subqueries**. They identify frequent customers, display customer order details, and calculate total purchases. This approach improves business analysis, supports customer relationship management, and enables organizations to make informed decisions based on customer purchasing patterns.

Q7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into Fourth Normal Form (4NF), explaining the reasoning.

Scenario Analysis

A company maintains employee information in a single relation containing multiple skills and multiple projects for each employee. Since an employee can possess several skills and simultaneously work on several projects, the relation contains **multi-valued dependencies (MVDs)**. These independent multi-valued attributes cause data redundancy, increase storage requirements, and lead to insertion, update, and deletion anomalies.

The objective is to analyze the relation, identify the multi-valued dependencies, and decompose it into **Fourth Normal Form (4NF)** to eliminate redundancy while preserving data integrity.

Given Relation

Employee (EmployeeID, EmployeeName, Skill, Project)

Sample Data

EmployeeID	EmployeeName	Skill	Project
E101	Arun	Java	Banking
E101	Arun	Java	E-Commerce
E101	Arun	Python	Banking
E101	Arun	Python	E-Commerce
E102	Priya	SQL	Healthcare
E102	Priya	SQL	Inventory

Analysis of the Relation

An employee may have:

- Multiple Skills
- Multiple Projects

The skills and projects are independent of each other.

Multi-Valued Dependencies

EmployeeID $\twoheadrightarrow$ Skill

EmployeeID $\twoheadrightarrow$ Project

This means:

- One employee can have many skills.
- One employee can work on many projects.
- Skills do not determine projects.
- Projects do not determine skills.

Therefore, the relation contains **Multi-Valued Dependencies (MVDs)**.

Problems in the Existing Relation

The relation has several drawbacks:

- Employee information is repeated.
- Skill values are repeated for every project.
- Project values are repeated for every skill.
- High storage redundancy.
- Update anomaly.
- Insertion anomaly.
- Deletion anomaly.
- Difficult to maintain.

Hence, the relation **does not satisfy Fourth Normal Form (4NF)**.

Decomposition into 4NF

To remove the multi-valued dependencies, split the relation into separate tables.

Employee Table

EmployeeID	EmployeeName
E101	Arun
E102	Priya

Primary Key: EmployeeID

Employee_Skill Table

EmployeeID	Skill
E101	Java
E101	Python
E102	SQL

Primary Key: (EmployeeID, Skill)

Foreign Key: EmployeeID references Employee(EmployeeID)

Employee_Project Table

EmployeeID	Project
E101	Banking
E101	E-Commerce
E102	Healthcare
E102	Inventory

Primary Key: (EmployeeID, Project)

Foreign Key: EmployeeID references Employee(EmployeeID)

Final 4NF Schema

Employee

(EmployeeID **PK**, EmployeeName)

Employee_Skill

(EmployeeID **FK**, Skill)

Primary Key: (**EmployeeID, Skill**)

Employee_Project

(EmployeeID **FK**, Project)

Primary Key: (**EmployeeID, Project**)

Why the Decomposition Satisfies 4NF

The decomposed relations satisfy **Fourth Normal Form** because:

- Each table contains only one independent multi-valued fact.
- No non-trivial multi-valued dependency exists.
- Every multi-valued dependency has a super key as its determinant.
- Redundant combinations of skills and projects are eliminated.

Advantages of 4NF

- Eliminates data redundancy.
- Removes multi-valued dependencies.
- Prevents insertion, update, and deletion anomalies.
- Improves data consistency.
- Reduces storage requirements.
- Simplifies database maintenance.
- Supports efficient query processing.
- Provides a scalable database design.

Justification

The original relation stored employee skills and projects together, causing repeated combinations of skills and projects. By decomposing the relation into **Employee**, **Employee_Skill**, and **Employee_Project**, each table stores only one type of independent information. This removes multi-valued dependencies, eliminates redundancy, and maintains referential integrity using primary and foreign keys. The resulting schema satisfies **Fourth Normal Form (4NF)** and is suitable for efficient database management.

Conclusion

The original Employee relation contained **multi-valued dependencies**, resulting in redundant data and database anomalies. After decomposition into **Employee**, **Employee_Skill**, and **Employee_Project** tables, all independent multi-valued attributes are separated. The final schema satisfies **Fourth Normal Form (4NF)**, eliminates redundancy, improves data integrity, and provides an efficient, scalable, and maintainable database structure for real-world applications.

Q8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

Scenario Analysis

A database stores employee information, but the designer failed to apply appropriate integrity constraints. As a result, duplicate records, invalid values, missing information, and broken relationships exist in the database. These inconsistencies reduce data accuracy and reliability.

The objective is to evaluate the schema, identify the problems caused by improper constraints, and suggest suitable **integrity constraints** to ensure data consistency, accuracy, and reliability.

Existing Schema

Employee Table

EmployeeID	EmployeeName	DepartmentID	Salary	Email
101	Arun	1	50000	arun@gmail.com
101	Arun	2	-1000	arun@gmail.com
NULL	Priya	3	45000	priya@gmail.com
103	Rahul	8	60000	rahul@gmail.com

Department Table

DepartmentID	DepartmentName
1	HR
2	IT
3	Finance

Problems Identified

The schema contains several inconsistencies:

- Duplicate Employee IDs.
- NULL values in the primary key.
- Negative salary values.
- Invalid DepartmentID that does not exist.
- Duplicate email addresses.
- Missing validation rules.
- Weak referential integrity.

These problems occur because appropriate integrity constraints were not implemented.

Suggested Integrity Constraints

Primary Key Constraint

The **EmployeeID** must uniquely identify every employee and should never be NULL.

```
mysql> CREATE TABLE Employee
-> (
-> EmployeeID INT PRIMARY KEY,
-> EmployeeName VARCHAR(50),
-> DepartmentID INT,
-> Salary DECIMAL(10,2),
-> Email VARCHAR(100)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Purpose

- Prevents duplicate employee records.
- Does not allow NULL values.
- Ensures every employee has a unique identity.

Foreign Key Constraint

The DepartmentID should reference an existing department.

```
mysql> FOREIGN KEY (DepartmentID)
-> REFERENCES Department(DepartmentID);
```

Purpose

- Prevents invalid department values.
- Maintains referential integrity.
- Ensures every employee belongs to a valid department.

NOT NULL Constraint

Mandatory fields should never be left empty.

```
mysql> EmployeeName VARCHAR (50) NOT NULL,  
-> Salary DECIMAL (10,2) NOT NULL
```

Purpose

- Ensures essential information is always entered.
- Prevents incomplete records.

UNIQUE Constraint

Each employee email should be unique.

```
mysql> Email VARCHAR(100) UNIQUE|
```

Purpose

- Prevents duplicate email addresses.
- Maintains unique employee contact information.

CHECK Constraint

```
mysql> Salary should always be positive.  
-> Salary DECIMAL(10,2)  
-> CHECK (Salary > 0)  
-> Another example:  
-> CHECK (DepartmentID BETWEEN 1 AND 3)
```

Purpose

- Prevents invalid salary values.
- Ensures only valid department IDs are accepted.

DEFAULT Constraint

Assign a default salary if no salary is entered.

```
mysql> Salary DECIMAL(10,2)  
-> DEFAULT 25000
```

Purpose

- Automatically assigns a valid default value.
- Reduces missing information.

Improved Employee Table

```
mysql> CREATE TABLE Employee
-> (
-> EmployeeID INT PRIMARY KEY,
-> EmployeeName VARCHAR(50) NOT NULL,
-> DepartmentID INT,
-> Salary DECIMAL(10,2) CHECK (Salary > 0),
-> Email VARCHAR(100) UNIQUE,
-> FOREIGN KEY (DepartmentID)
-> REFERENCES Department(DepartmentID)
-> );
```

Advantages of Integrity Constraints

- Prevents duplicate records.
- Eliminates invalid data entry.
- Maintains referential integrity.
- Improves data accuracy.
- Ensures consistency across related tables.
- Prevents insertion, update, and deletion anomalies.
- Simplifies database maintenance.
- Improves overall database reliability.

Justification

The proposed integrity constraints ensure that only valid and consistent data can be stored in the database. **Primary Key** guarantees uniqueness, **Foreign Key** maintains relationships between tables, **NOT NULL** ensures mandatory fields are filled, **UNIQUE** prevents duplicate values, **CHECK** validates business rules such as positive salary, and **DEFAULT** assigns predefined values when necessary. Together, these constraints eliminate inconsistencies and improve the quality and reliability of the database.

Conclusion

The original schema suffered from inconsistent data due to the absence of proper integrity constraints. By implementing **Primary Key, Foreign Key, NOT NULL, UNIQUE, CHECK, and DEFAULT** constraints, the database becomes more secure, accurate, and reliable. These constraints enforce valid data entry, preserve referential integrity, reduce anomalies, and ensure efficient database management. The improved schema is suitable for real-world database applications and satisfies the requirements of a well-designed relational database.

Q9. For a given relational schema, analyze the presence of join dependencies and decompose it into Fifth Normal Form (5NF) to remove redundancy.

Scenario Analysis

A company maintains information about **Suppliers, Products, and Projects** in a single relation. A supplier may supply multiple products, a product may be used in multiple projects, and a supplier may supply products for several projects. Storing all this information in one table creates **join dependencies**, resulting in data redundancy and maintenance problems.

The objective is to identify the join dependencies and decompose the relation into **Fifth Normal Form (5NF)** to eliminate redundancy while preserving all information.

Given Relation:

Supplier_Product_Project (SupplierID, ProductID, ProjectID)

Sample Data

SupplierID	ProductID	ProjectID
S1	P1	PR1
S1	P2	PR1
S2	P1	PR2
S2	P2	PR2

Analysis of Join Dependency

The relation has the following independent relationships:

- Supplier supplies Products.
- Supplier works with Projects.
- Product is used in Projects.

The relation contains a **Join Dependency (JD)** because the complete relation can be reconstructed by joining these smaller relations.

Join Dependency:

$(\text{SupplierID}, \text{ProductID}, \text{ProjectID}) \rightarrow (\text{SupplierID}, \text{ProductID})$
 $(\text{SupplierID}, \text{ProjectID})$
 $(\text{ProductID}, \text{ProjectID})$

Since the relation can be losslessly decomposed into smaller relations, it **does not satisfy Fifth**

Normal Form (5NF).

Problems in the Existing Relation

The original relation has the following issues:

- Data redundancy due to repeated combinations.
- Repeated supplier information.
- Repeated product information.
- Repeated project information.
- Update anomaly.
- Insertion anomaly.
- Deletion anomaly.
- High storage requirements.

Decomposition into Fifth Normal Form (5NF)

The relation is decomposed into three smaller relations.

Supplier_Product Table

SupplierID	ProductID
S1	P1
S1	P2
S2	P1
S2	P2

Primary Key: (SupplierID, ProductID)

Supplier_Project Table

SupplierID	ProjectID
S1	PR1
S2	PR2

Primary Key: (SupplierID, ProjectID)

Product_Project Table

ProductID	ProjectID
P1	PR1
P2	PR1
P1	PR2
P2	PR2

Primary Key: (ProductID, ProjectID)

Final 5NF Schema

Supplier_Product

(SupplierID **PK**, ProductID **PK**)

Supplier_Project

(SupplierID **PK**, ProjectID **PK**)

Product_Project

(ProductID **PK**, ProjectID **PK**)

Verification of 5NF

The decomposed relations satisfy **Fifth Normal Form (5NF)** because:

- Every join dependency is represented by candidate keys.
- No redundant tuples exist.
- The original relation can be reconstructed using natural joins without losing information.
- There are no remaining join anomalies.

Advantages of 5NF

- Eliminates join redundancy.
- Removes insertion, update, and deletion anomalies.
- Improves data consistency.
- Reduces unnecessary storage.
- Preserves lossless decomposition.
- Simplifies database maintenance.
- Supports efficient database design.
- Suitable for complex relational databases.

Justification

The original relation contained **join dependencies**, causing repeated combinations of suppliers, products, and projects. By decomposing the relation into **Supplier_Product**, **Supplier_Project**, and **Product_Project**, each table stores only one independent relationship. The original relation can be reconstructed through natural joins without losing any information. This decomposition eliminates redundancy, improves consistency, and satisfies the requirements of **Fifth Normal Form (5NF)**.

Conclusion

The given relational schema contained **join dependencies**, leading to redundancy and database anomalies. By decomposing the relation into **Supplier_Product**, **Supplier_Project**, and **Product_Project** tables, the schema satisfies **Fifth Normal Form (5NF)**. The new design removes redundancy, preserves all information through lossless joins, improves data integrity, and provides a scalable and efficient database suitable for real-world applications.

Q10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

Scenario Analysis

A company maintains a database containing customer and order information. The existing SQL queries retrieve data inefficiently by using unnecessary subqueries, repeated table scans, and missing indexes. As the database grows, query execution becomes slow, increasing response time and reducing overall system performance.

The objective is to analyze the inefficient queries and optimize them using **JOIN operations, indexing techniques, and query restructuring** to improve retrieval speed and database performance.

Database Schema

Customer Table

CustomerID	CustomerName	City
C101	Arun	Chennai
C102	Priya	Vellore
C103	Rahul	Bangalore

Primary Key: CustomerID

Orders Table

OrderID	CustomerID	OrderDate	Amount
O101	C101	2026-08-01	5000
O102	C101	2026-08-02	3500
O103	C102	2026-08-03	7000
O104	C103	2026-08-04	4500

Foreign Key: CustomerID references Customer(CustomerID)

Inefficient Query

The following query retrieves customer names using a subquery.

```
mysql> SELECT CustomerName
-> FROM Customer
-> WHERE CustomerID IN
-> (
->     SELECT CustomerID
->     FROM Orders
-> );
```

```
+-----+
| CustomerName |
+-----+
| Arun         |
| Priya        |
| Rahul        |
+-----+
```

```
3 rows in set (0.02 sec)
```

Problems

- Uses a subquery unnecessarily.
- May perform multiple scans on large tables.
- Slower execution for large databases.
- Less efficient than using JOIN.

Optimized Query Using JOIN

```
mysql> SELECT DISTINCT
-> C.CustomerID,
-> C.CustomerName,
-> O.OrderID,
-> O.Amount
-> FROM Customer C
-> INNER JOIN Orders O
-> ON C.CustomerID = O.CustomerID;
```

CustomerID	CustomerName	OrderID	Amount
C101	Arun	0101	5000.00
C101	Arun	0102	3500.00
C102	Priya	0103	7000.00
C103	Rahul	0104	4500.00

4 rows in set (0.00 sec)

Explanation

- **INNER JOIN** directly combines Customer and Orders tables.
- Retrieves matching records efficiently.
- **DISTINCT** avoids duplicate rows.
- Improves execution speed compared to nested subqueries.

Creating an Index

To improve query performance, create an index on the foreign key.

```
mysql> CREATE INDEX idx_customer
-> ON Orders(CustomerID);
Query OK, 0 rows affected (0.09 sec)
```

Benefits of Indexing

- Faster searching.
- Reduces table scanning.
- Improves JOIN performance.
- Enhances retrieval speed.

Query Restructuring

Instead of using multiple nested queries, use aggregation directly.

Inefficient Query

```
mysql> SELECT CustomerID
-> FROM Orders
-> GROUP BY CustomerID
-> HAVING COUNT(*) > 1;
+-----+
| CustomerID |
+-----+
| C101       |
+-----+
1 row in set (0.02 sec)
```

Optimized Query

```
mysql> SELECT
-> C.CustomerID,
-> C.CustomerName,
-> COUNT(O.OrderID) AS TotalOrders
-> FROM Customer C
-> JOIN Orders O
-> ON C.CustomerID = O.CustomerID
-> GROUP BY C.CustomerID, C.CustomerName;
+-----+-----+-----+
| CustomerID | CustomerName | TotalOrders |
+-----+-----+-----+
| C101       | Arun         | 2           |
| C102       | Priya        | 1           |
| C103       | Rahul        | 1           |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

This query retrieves customer details and the total number of orders in a single operation.

Performance Improvements

Technique	Improvement
INNER JOIN	Faster retrieval than nested subqueries
INDEX	Speeds up searching and JOIN operations
GROUP BY	Efficient aggregation of records
DISTINCT	Removes duplicate rows
Query Restructuring	Reduces unnecessary table scans

Advantages of Optimization

- Faster query execution.
- Reduces CPU and memory usage.
- Improves response time.
- Minimizes table scans.
- Efficient use of indexes.
- Better scalability for large databases.
- Improves overall database performance.
- Enhances user experience.

Justification

The optimized solution replaces inefficient subqueries with **JOIN** operations, which are more efficient for retrieving related data. Creating an **index** on the foreign key allows the database to locate matching records quickly, reducing search time. Query restructuring eliminates unnecessary operations and combines data retrieval into a single optimized query. These techniques significantly improve performance while maintaining data accuracy and consistency.

Conclusion

The original SQL queries were inefficient because they relied on unnecessary subqueries and lacked indexing. By optimizing the queries using **INNER JOIN**, **INDEX**, and **query restructuring**, the system achieves faster data retrieval, reduced execution time, improved scalability, and better resource utilization. These optimization techniques are widely used in real-world database systems to ensure efficient and reliable performance.